



NP-Completeness (7.4)

COT 4210 Discrete Structures II
Summer 2025
Department of Computer Science
Dr. Steinberg

Satisfiability and NP-Completeness

- We've demonstrated that $\text{SAT} \in \text{NP}$. Now we're going to demonstrate one of the most important results in computer science: the **Cook-Levin Theorem**.
- SAT is **NP-Complete**. *Every language in NP is polynomially reducible to SAT.*
- A reduction this generic seems doomed to either fail or require an entire book to go through, but it's actually straightforward. (Okay, relatively straightforward.) Recall two things:
 - When we wanted to show that PCP was undecidable, we encoded the computation history of a Turing machine into it.
 - SAT is just composed of a set of Boolean formulas. It's easy to imagine any computer running it, because a computer is just a set of logic gates. **That goes both ways!**
- We are going to show that **SAT can represent the computation history of any nondeterministic Turing machine**, in such a way that a satisfiable assignment exists if and only if the NDTM can accept. Since any NP problem can be solved in polynomial time by some NDTM, the NP-completeness of SAT immediately follows.

The Proof Idea

- An NDTM M accepts w if and only if, run on w , one of its nondeterministic branches becomes an accepting computation history.
- An accepting computation history is a sequence of configurations where:
 - The first configuration is the initial configuration of M on w .
 - Every subsequent configuration is yielded by the previous configuration – that is, it's a legal move for M .
 - The final configuration is an accepting configuration - that is, its state is q_{ACCEPT} .
- We can use Boolean logical formulas easily to require the first and last of a configuration history, and the middle one with a bit of thought. However, first we need to represent the configuration history in the first place.

Tableaus

A **tableau** is an array of tape alphabet symbols.

- It represents a configuration history of **one branch** of our NDTM's nondeterminism.
- If the NDTM runs in n^k time, the tableau is an $(n^k \times n^k)$ tableau.
 - It's big enough downward because, well, the TM runs in n^k .
 - ...and rightward because the TM can only *count* to n^k .
- We assume that every configuration starts and ends with a # symbol.
- We think of our tableau as looking like this in the “beginning”: the starting configuration across the top, and the other configurations blank.
 - (We quote “beginning” because SAT isn't really a stateful algorithm, but just go with it for now.)
- But we've assumed that we can “represent” alphabet symbols. How do we do that, in SAT?

#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#	$\uparrow$ n^k $\downarrow$
#									#	
#									#	
#									#	
#									#	
#									#	
#									#	
#									#	
#									#	
#									#	
$\leftarrow n^k \rightarrow$										

Tableaus

- Number the rows and columns of the tableau, then think of a *SAT*-style variable in SAT corresponding to an individual cell:

$$x_{i,j}$$

- So far, so good.
 - We can pretty easily think of that as encoding the binary value of a cell in our tableau.
 - But we need to go a little farther than that.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Encoding the Tableau: Basics

- Consider a cell and corresponding variable:

$$x_{i,j}$$

- We need to encode more than binary into this. Consider a set comprised of:

- The tape alphabet
- The state set
- The separator character

$$C = \Gamma \cup Q \cup \{ \# \}$$

- Now rethink our cell variable:

$$x_{i,j,c}$$

- And now, **turning this variable on** corresponds to **setting cell $(i, j) = c$** , for some $c \in C$.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Encoding the Tableau: Cells

- Consider our tableau alphabet:

$$C = \Gamma \cup Q \cup \{ \# \}$$
- Consider a cell and corresponding variable:

$$x_{i,j,c}$$
- Now we need to make sure the tableau is consistently encoded.
 - Create a clause for **each cell** (i, j) .

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c, d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

- The left demands $x_{i,j,c}$ be true for **some** c .
- The right demands $x_{i,j,c}$ be true for **only one** c .

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Encoding the Tableau: The Tableau

Tableau alphabet: $C = \Gamma \cup Q \cup \{ \# \}$

Cell variable: $x_{i,j,c}$

Create an encoding clause for each cell (i, j).

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c, d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

Now repeat the clause across the tableau.

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

This is our *cell formula*. It ensures that each cell in the tableau is assigned a single symbol.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Encoding the Tableau: Complexity

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c,d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

We can create the single-cell encoding formula in polynomial time with a $|C|^2$ iteration.

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

We can create the *entire* cell formula in polynomial time with an n^{2k} iteration around that.

So we can say that ϕ_{cells} **is satisfied by, and only by, a properly encoded tableau, and is created in polynomial time.**

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Starting and Accepting

- Starting and accepting are (comparatively) easy.
- To start, take the start configuration padded to n^k length with blanks...

$S = \#q_0w_1w_2...w_n\Box...\Box\#$ so that $|S| = n^k$

- ...and **require the first row be equal to the start configuration:**

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

- Then to accept, just **require an accept state somewhere in the tableau.**

$$\phi_{\text{accept}} = \bigvee_{1 \leq i,j \leq n^k} [x_{i,j,q_A}]$$

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\Box$	...	$\Box$	#
2	#									#
3	#									#
4	#									#
5	#	w_1	w_2	...	q_A	...	$\Box$	...	$\Box$	#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Starting and Accepting

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i,j \leq n^k} [x_{i,j,q_A}]$$

We can generate the start and accept formulas in n^k and $(n^k)^2$ time, both polynomial.

So now we can say that:

ϕ_{start} is satisfied by, and only by, a tableau with the starting configuration of M on w encoded as its first row, and is created in polynomial time.

...and...

ϕ_{accept} is satisfied by, and only by, a tableau encoding an accepting configuration as one of its rows, and is created in polynomial time.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	w_1	w_2	...	w_n	$\square$	...	$\square$	#
2	#									#
3	#									#
4	#									#
5	#	z_1	z_2	...	q_A	...	$\square$	...	$\square$	#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Starting and Acceptance

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

We can generate the starting configuration in $(n^k)^2$ time, both polynomial.

So now we can say that:

ϕ_{start} is satisfied by, and the starting configuration is created by its first row, and is created by...

ϕ_{accept} is satisfied by, and encoding an accepting configuration is created by its rows, and is created by...

IMPORTANT! This example assumes that our Turing Machine reaches an acceptance state before the n^k step. It will loop through the entire tableau in order to find the acceptance state.

$$\phi_{\text{accept}} = \bigvee_{1 \leq i,j \leq n^k} [x_{i,j,q_A}]$$

However, if we assumed that the tableau just fills in the blank rows with copies of the last configuration that contains the acceptance state. Then we can apply the following expression since the last row will contain a configuration.

$$\phi_{\text{accept}} = \bigvee_{1 \leq j \leq n^k} [x_{n^k,j,q_A}]$$

5	6	7	8	9	10
..	w_n	$\square$	...	$\square$	#
					#
					#
					#
q_A	...	$\square$	...	$\square$	#
					#
					#
					#
					#
					#
					#

Transitions

Now, for transitions. Recall the dominoes we used to encode transitions for PCP.

We're going to do basically the same thing, except instead of dominoes, we use *windows*.

- A given 2x3 **window** is **legal** if it does not violate our machine's transition function.
- Given the linear sets of states and tape symbols, and the finite size of 2x3 windows, we can make a **polynomial-sized set of all legal windows**, the same way we did for PCP dominoes.

Let a sequence $A = (a_1, \dots, a_6)$ be a 2x3 window, with a_1 the top left cell, a_2 the top middle, etc.

- We say that **A is legal** if it **represents a legal window**.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	a	b	c	a	□	□	□	#
2	#	b	q_1	b	c	a	□	□	□	#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Transitions

- A given 2x3 **window** is **legal** if it does not violate our machine's transition function. We have a **polynomial-sized set of all legal windows**.
- Let a sequence $A = (a_1, \dots, a_6)$ be a 2x3 window. **A is legal** if it **represents a legal window**.
- Now we can come up with a formula to say that the window top-centered at cell (i, j) is legal.

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \right]$$

- **Don't be intimidated by this formula!**
 - It's just **counting off the six cells of the window** and demanding that each be **equal to the corresponding cell in some legal window**.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	a	b	c	a	$\square$	$\square$	$\square$	#
2	#	b	q_1	b	c	a	$\square$	$\square$	$\square$	#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Transitions

- A given 2x3 **window** is **legal** if it does not violate our machine's transition function.
- We have a **polynomial-sized set of all legal windows**.
- Let a sequence $A = (a_1, \dots, a_6)$ be a 2x3 window. **A is legal** if it **represents a legal window**.

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \right]$$

- Since we have a polynomial number of legal windows, this formula is also polynomial. So we can say:
- $\phi_{\text{legal}}(i, j)$ is satisfied by, and only by, a tableau whose window top-centered at (i, j) is legal; and is created in polynomial time.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	a	b	c	a	$\square$	$\square$	$\square$	#
2	#	b	q_1	b	c	a	$\square$	$\square$	$\square$	#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Windows and Configurations

Consider any **upper** and **lower** configuration in the tableau, so that the lower configuration is the one immediately below – that is, following – the upper.

If all the windows top-centered on cells in the upper configuration are legal, then:

- The legality of the windows that don't involve the state symbol easily ensures the legality of the configuration below them.
- The window top-centered on the state symbol in the upper configuration is sufficient to ensure that the state symbol in the lower configuration makes a legal move.
 - If the state symbol is moving to the right, it acts like the relevant 2-wide domino in PCP.
 - If the state symbol is moving to the left, it acts like the relevant 3-wide domino in PCP.

The upper configuration yields the lower one if and only if all the windows top-centered on cells in the upper configuration are legal – and that holds all the way down the tableau.

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	a	b	c	a	□	□	□	#
2	#	b	q_1	b	c	a	□	□	□	#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Windows and Configurations

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[\begin{array}{l} x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge \\ x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \end{array} \right]$$

$\phi_{\text{legal}}(i, j)$ is satisfied by, and only by, a tableau whose window top-centered at (i, j) is legal; and is created in polynomial time.

An upper configuration yields a lower one iff all the windows top-centered within the upper are legal.

- This holds all the way down the tableau.

Then we have:

$$\phi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k, \\ 1 < j < n^k}} \phi_{\text{legal}}(i, j)$$

And can say ϕ_{move} is satisfied by, and only by, a tableau that **does not violate the machine's transition function; and is created in polynomial time.**

	1	2	3	4	5	6	7	8	9	10
1	#	q_0	a	b	c	a	$\square$	$\square$	$\square$	#
2	#	b	q_1	b	c	a	$\square$	$\square$	$\square$	#
3	#									#
4	#									#
5	#									#
6	#									#
7	#									#
8	#									#
9	#									#
10	#									#

Pulling It Together

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} [x_{i,j,q_A}]$$

$$\phi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k, \\ 1 < j < n^k}} \phi_{\text{legal}}(i, j)$$

$$\phi_{\text{NDTM}} = (\phi_{\text{cells}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}})$$

We have:

- ϕ_{cells} is satisfied by, and only by, a properly encoded tableau.
- ϕ_{start} is satisfied by, and only by, a tableau with the starting configuration of M on w encoded as its first row.
- ϕ_{accept} is satisfied by, and only by, a tableau encoding an accepting configuration as one of its rows.
- ϕ_{move} is satisfied by, and only by, a tableau that does not violate the machine's transition function.
- All are created in polynomial time.

Then ϕ_{NDTM} is satisfied by, and only by, a tableau encoding an accepting computation history of M on w , and is created in polynomial time.

Corollary: 3SAT is NP-Complete

Given that SAT is NP-complete, it's relatively straightforward to demonstrate that 3SAT is.

- ...except that it's **not** a proof by reduction.

Recall that 3SAT satisfies Boolean 3cnf-formulas:

- A **cnf-formula** is a set of literals OR'd into clauses in turn AND'd into the formula.
- A **3cnf-formula** is a cnf-formula where all clauses have exactly three literals, such as:

$$\phi = (\bar{x} \vee y \vee \bar{z}) \wedge (x \vee y \vee \bar{z})$$

The temptation is to try to reduce SAT to 3SAT by generally reducing Boolean formulas to 3cnf.

- That doesn't work.**
- There is **not** a good way to generate cnf-formulas from arbitrary Boolean formulas.

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c,d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} [x_{i,j,q_A}]$$

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \right]$$

$$\phi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k, \\ 1 < j < n^k}} \phi_{\text{legal}}(i, j)$$

$$\phi_{\text{NDTM}} = (\phi_{\text{cells}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}})$$

Corollary: 3SAT is NP-Complete

Instead, we manipulate our formulas directly.

- ϕ_{accept} is a single clause.
- ϕ_{start} is a set of clauses.
- ϕ_{cells} is an **AND** of subformulas.
 - Each one has a single big **OR**, which is already a clause...
 - **AND** a bunch of **ANDs** of **ORs**, which pull out into clauses.
- ϕ_{move} is a **AND** of subformulas.
 - Each subformula is a large **OR** of **ANDs**.
 - Break these out by the distributive laws.
 - It's a **mess**, but a polynomial one in the size of the NDTM.

We now have a cnf-formula version of ϕ_{NDTM} .

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c,d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} [x_{i,j,q_A}]$$

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \right]$$

$$\phi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k, \\ 1 < j < n^k}} \phi_{\text{legal}}(i, j)$$

$$\phi_{\text{NDTM}} = (\phi_{\text{cells}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}})$$

Corollary: 3SAT is NP-Complete

We now have a cnf-formula version of ϕ_{NDTM} .

We want a 3cnf-formula. And for this part, direct reduction **does** work.

- For a one-clause (x) , write $(x \vee x \vee x)$.
- For a two-clause $(x \vee y)$, write $(x \vee y \vee x)$.
- For a three-clause, write it.
- For a four-clause $(w \vee x \vee y \vee z)$, write $(\alpha \vee w \vee x) \wedge (\bar{\alpha} \vee y \vee z)$.

And so on.

We now have a 3cnf-formula version of ϕ_{NDTM} ; 3SAT being NP-complete follows immediately.

$$\phi_{\text{encode}}(i, j) = \left[\left(\bigvee_{c \in C} x_{i,j,c} \right) \wedge \left(\bigwedge_{\substack{c,d \in C \\ c \neq d}} (\overline{x_{i,j,c}} \vee \overline{x_{i,j,d}}) \right) \right]$$

$$\phi_{\text{cells}} = \bigwedge_{1 \leq i, j \leq n^k} \phi_{\text{encode}}(i, j)$$

$$\phi_{\text{start}} = \bigwedge_{1 \leq j \leq n^k} [x_{1,j,s_k}]$$

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} [x_{i,j,q_A}]$$

$$\phi_{\text{legal}}(i, j) = \bigvee_{\substack{A=(a_1, \dots, a_6) \\ \text{is legal}}} \left[\begin{array}{l} x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge \\ x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6} \end{array} \right]$$

$$\phi_{\text{move}} = \bigwedge_{\substack{1 \leq i < n^k, \\ 1 < j < n^k}} \phi_{\text{legal}}(i, j)$$

$$\phi_{\text{NDTM}} = (\phi_{\text{cells}} \wedge \phi_{\text{start}} \wedge \phi_{\text{accept}} \wedge \phi_{\text{move}})$$



Acknowledgement

Some Notes and content come from Dr. Gerber, Dr. Hughes, and Mr. Guha's COT4210 class and the Sipser Textbook, *Introduction to the Theory of Computation*, 3rd ed., 2013